

# **ANÁLISE DE DESEMPENHO DE ALGORITMOS DE ORDENAÇÃO PARA AMBIENTE SOB ESTRESSE - ETAPA 3**

**Emanuel Rauter, Geancarlo Kozenieski, Tiago Elias Steinke**

## **AMBIENTE DE TESTE**

### **Hardware**

- Laptop AORUS 15P XD
- 11th Gen Intel i7-11800H 8 core (16 threads)
- 32 GB Ddr4 3200MHz Dual-channel (2x16 GB)
- 1TB M.2 2280 PCIe 4.0 NVMe

### **Código Fonte**

- C++

### **Sistema**

- Ubuntu 24.04.3
- 6.17.0-23-generic

### **Análise Dados**

- Pandas
- Jupiter Notebooks

## AMBIENTE DE TESTE

### Estrutura

- Datasets de tamanho  $10^7$
- 8 vetores característicos
- Execução: 1 algoritmo + 1 vetor
- 5 repetições de cada teste
- 4 configurações de estresse: nenhum, de memória, de CPU e ambas;
- 3 níveis de estresse: 0%, 50% e 100%
- Gerador de vetores em .bin
- Leitura de .bin e escrita em .csv

### Datasets

- Ordenados
- Inversamente Ordenados
- Totalmente Randômicos
- 1 número repetido
- Vetor somente de 1's a 5's
- Zigue-zague (Número baixo, número alto)
- Vetor com 1% de ruído na cauda
- Vetor onde 1% sofreu swaps aleatórios

## LEITURA DE DADOS

### Tempo

Biblioteca Chrono  
Medida em milissegundos

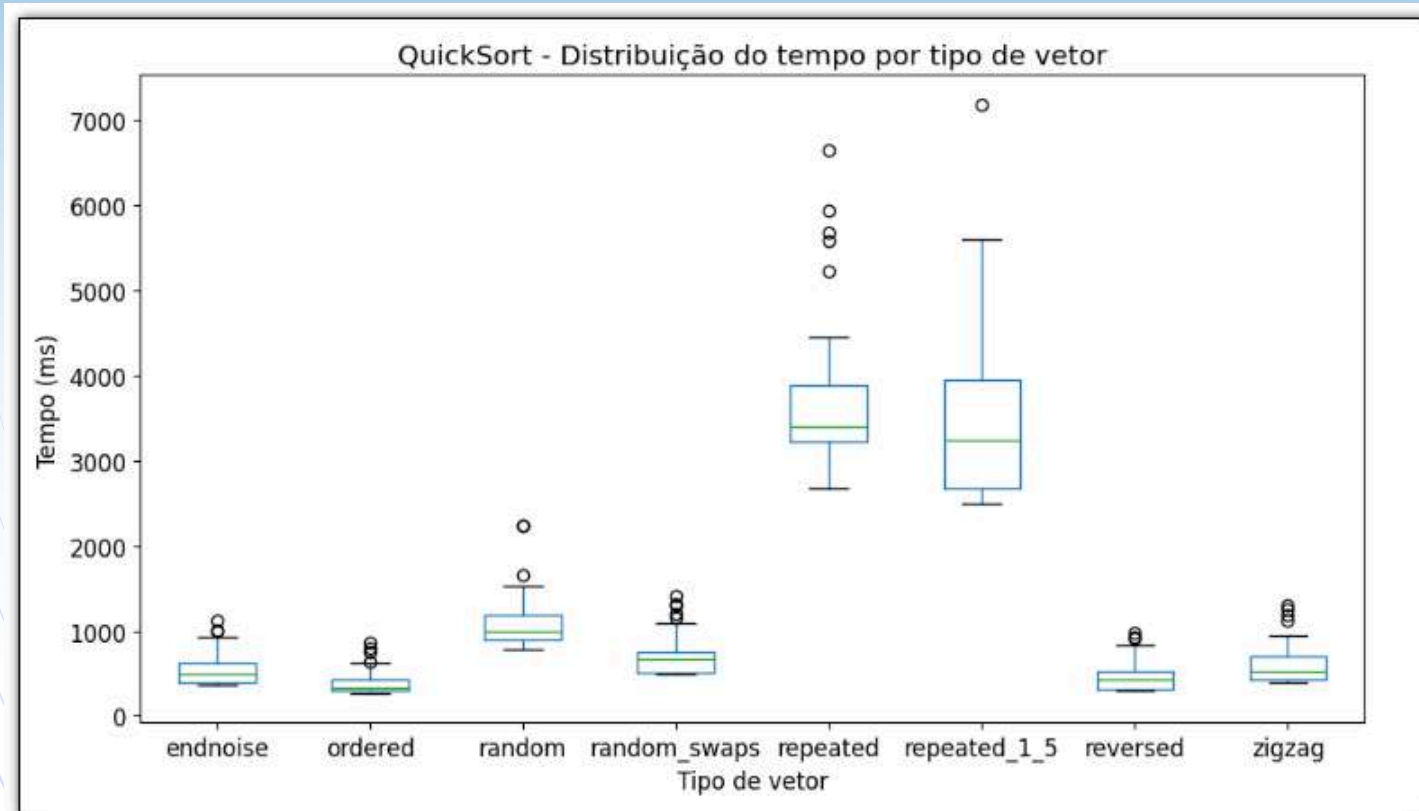
### Memória

Status do processo  
Medida em kilobytes

### Energia

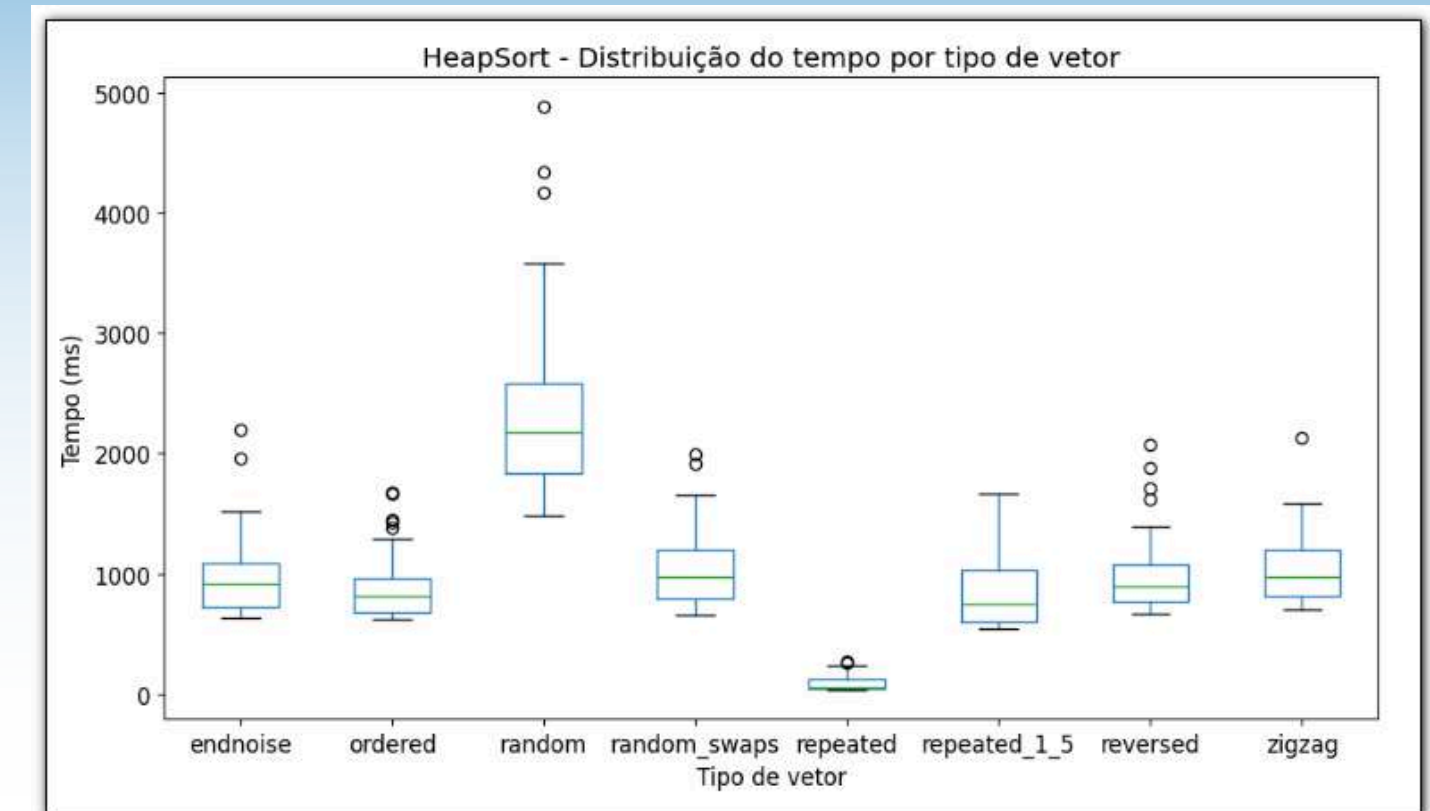
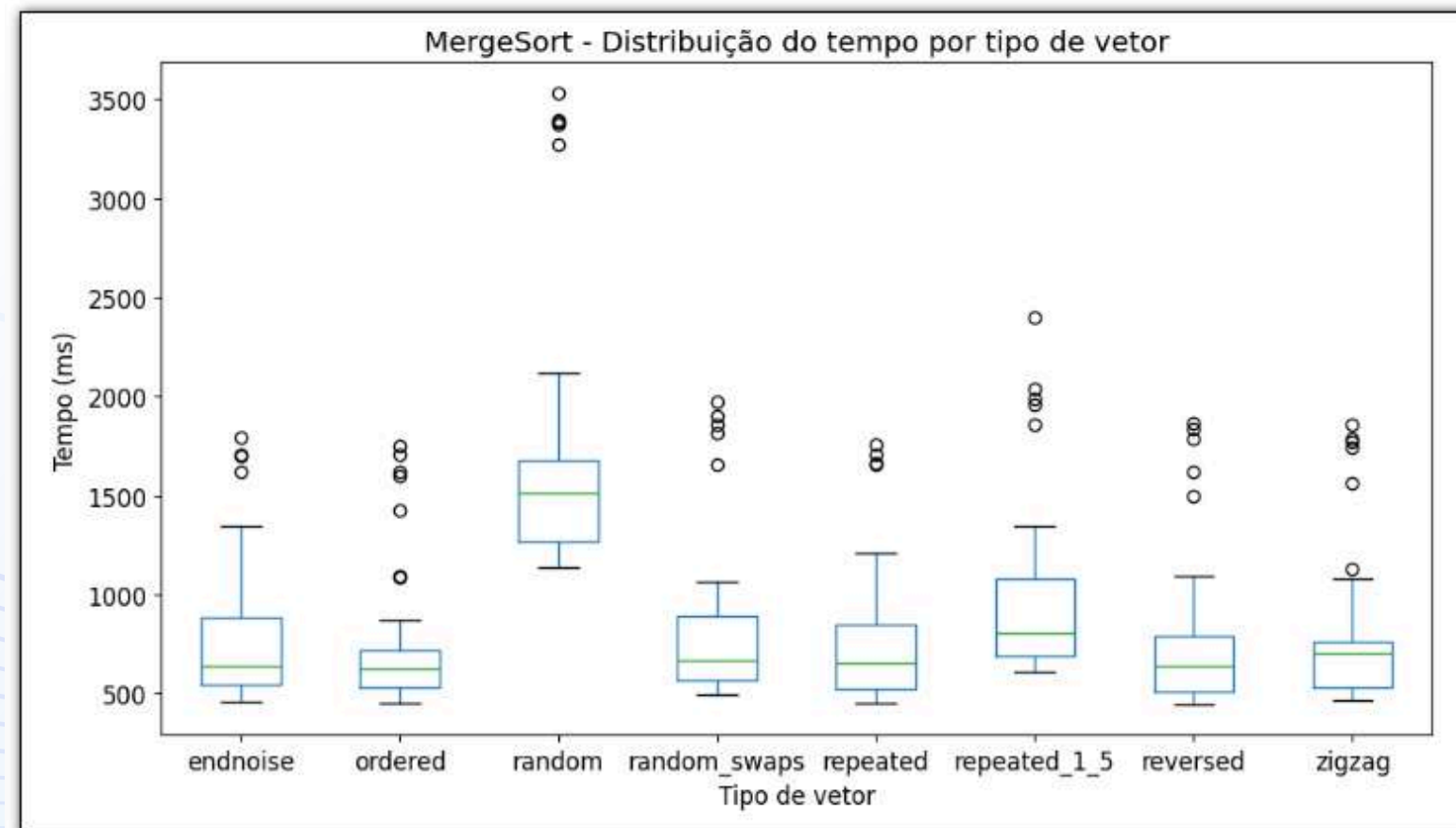
Intel Rapl  
Medida em Joules

# TEMPO INDIV.



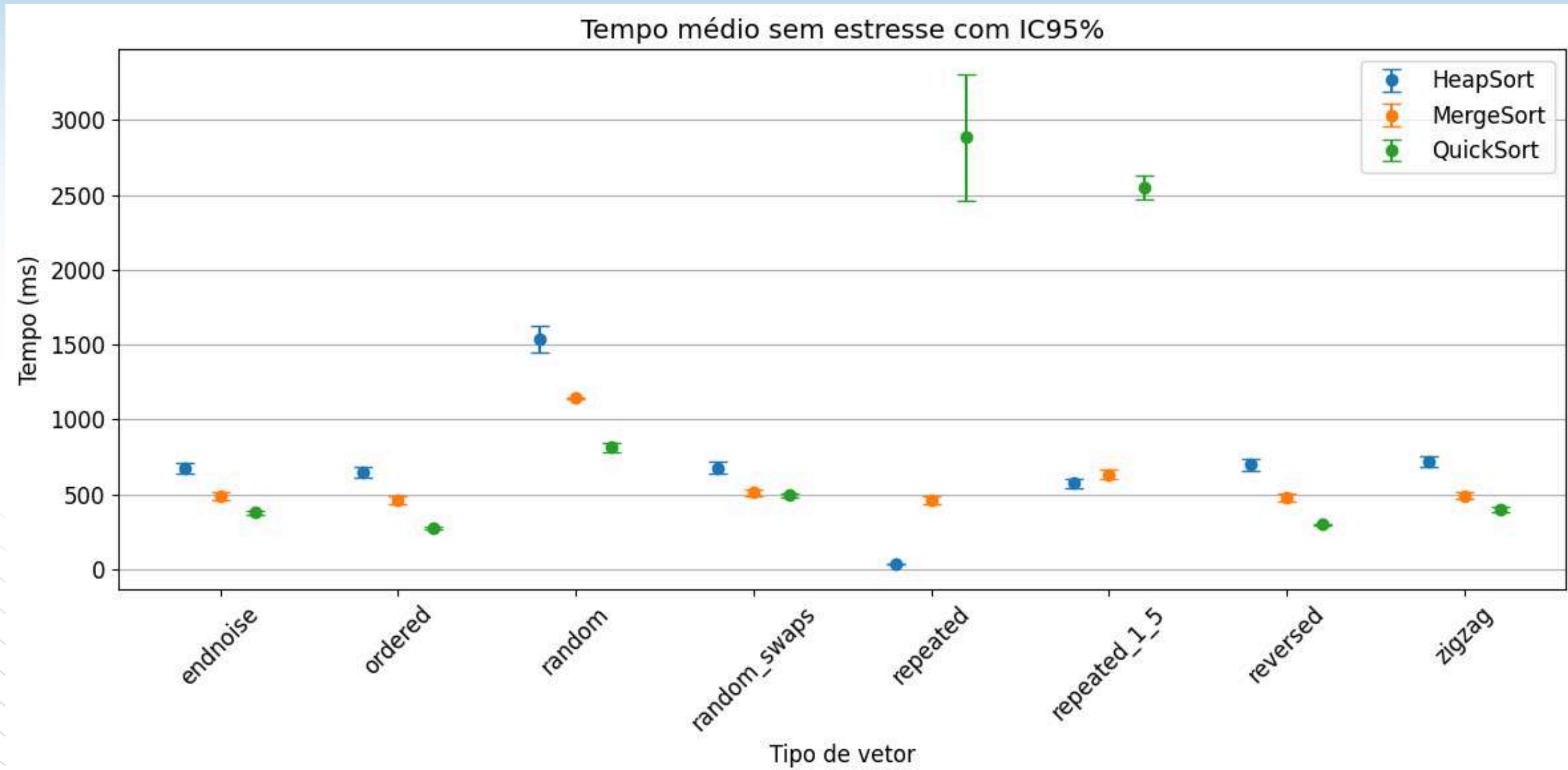
**QUICKSORT**

**MERGESORT**

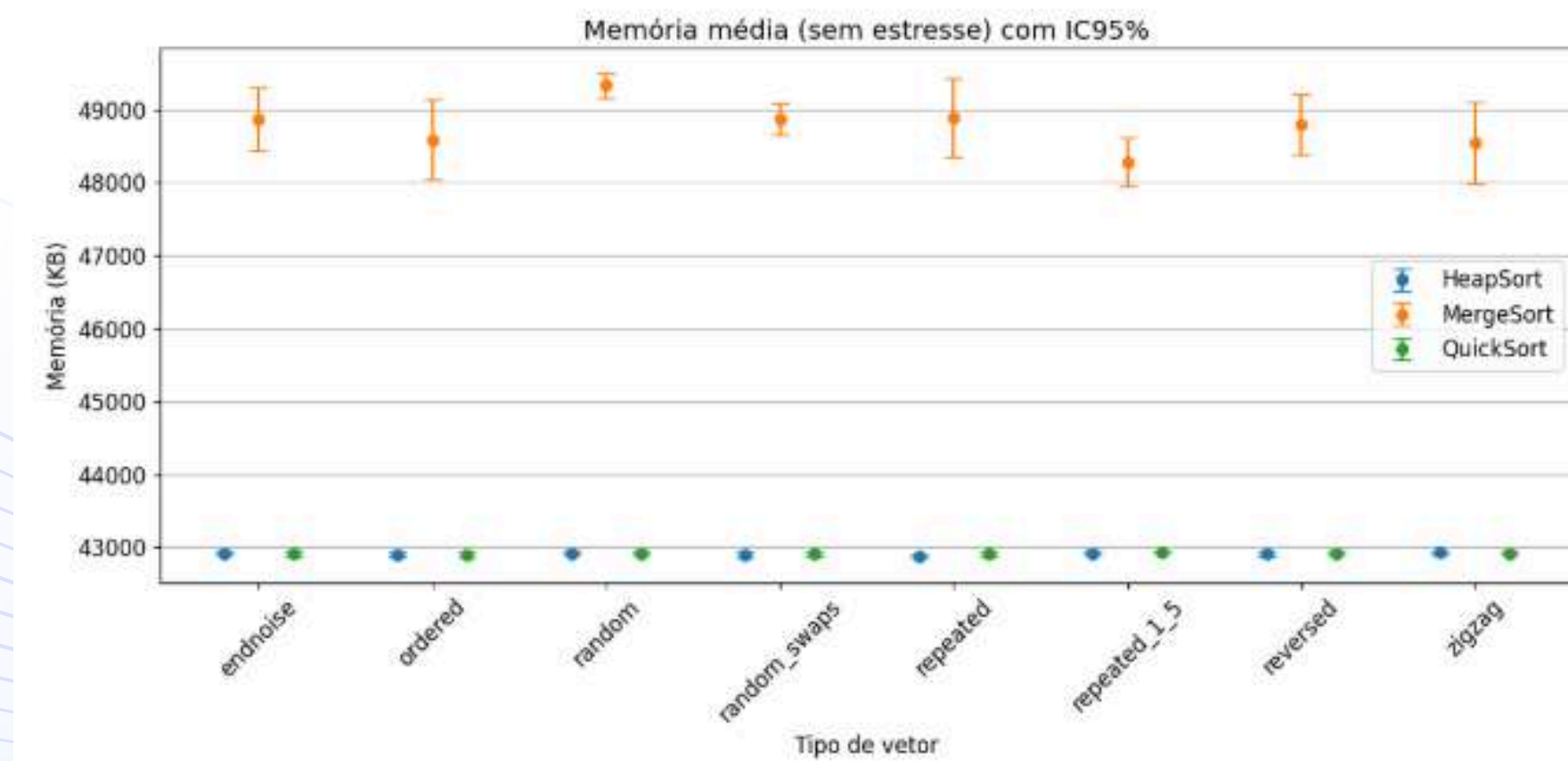
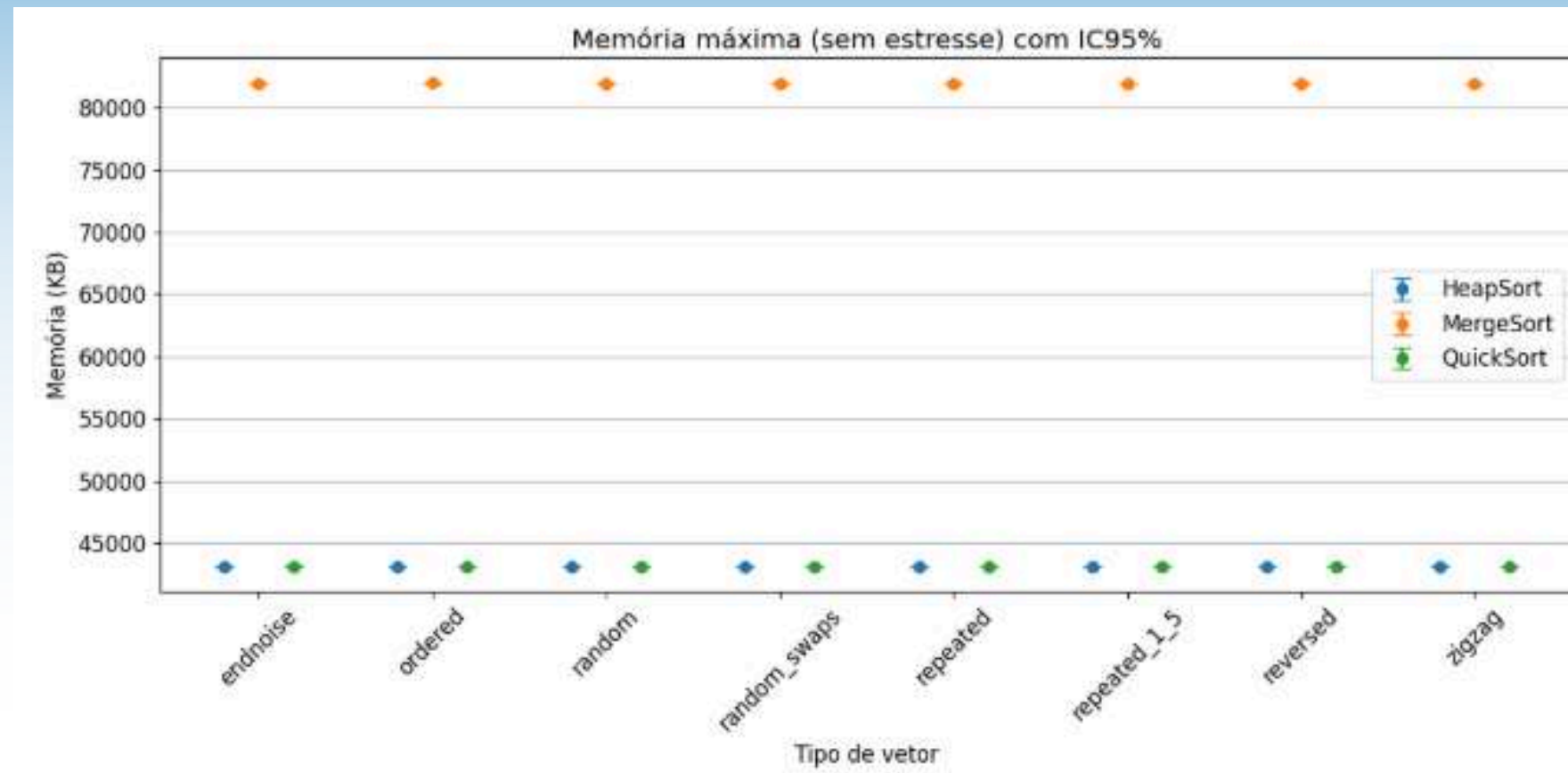


**HEAPSORT**

# TEMPO COMB.

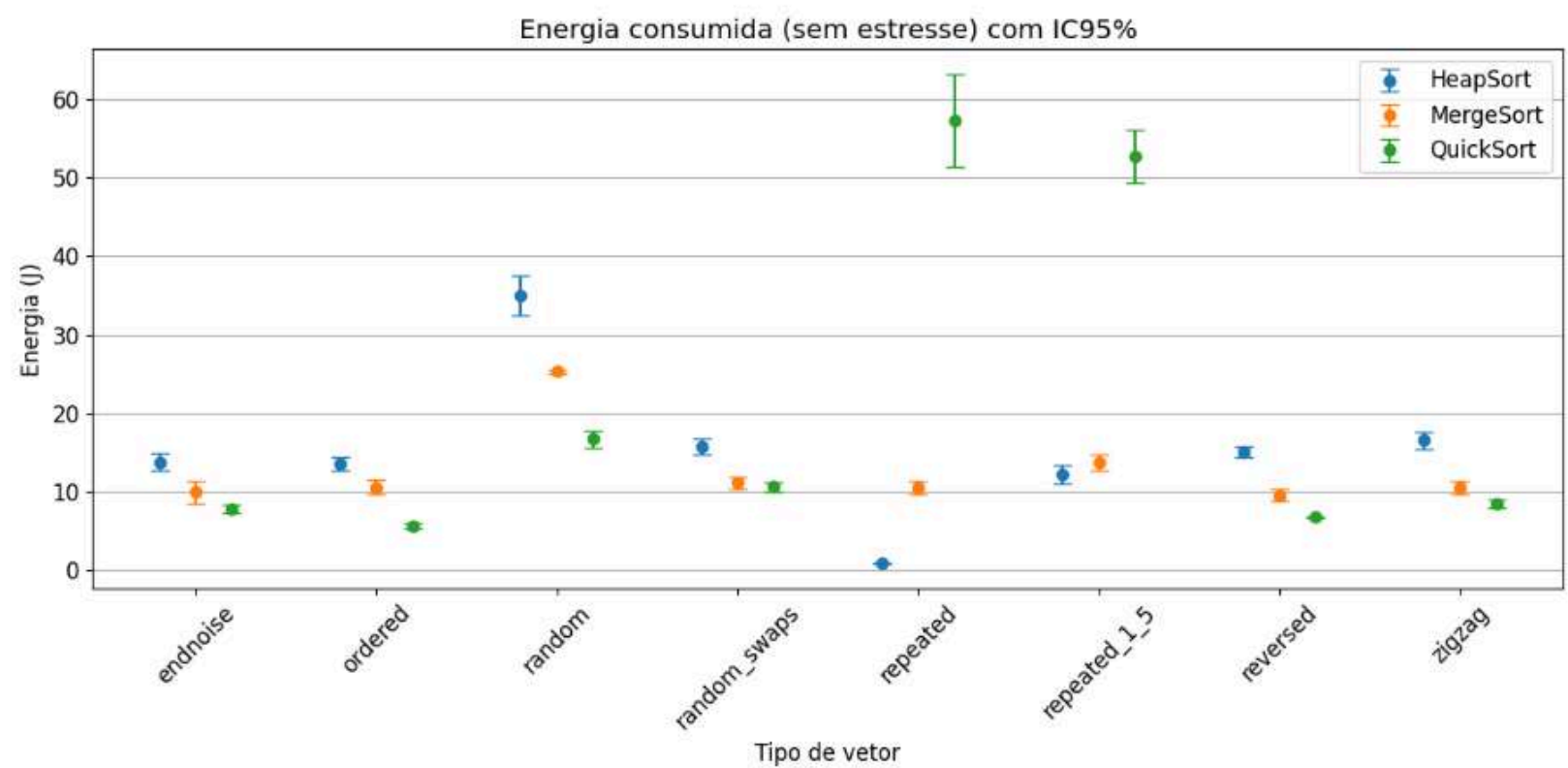


# MEMÓRIA

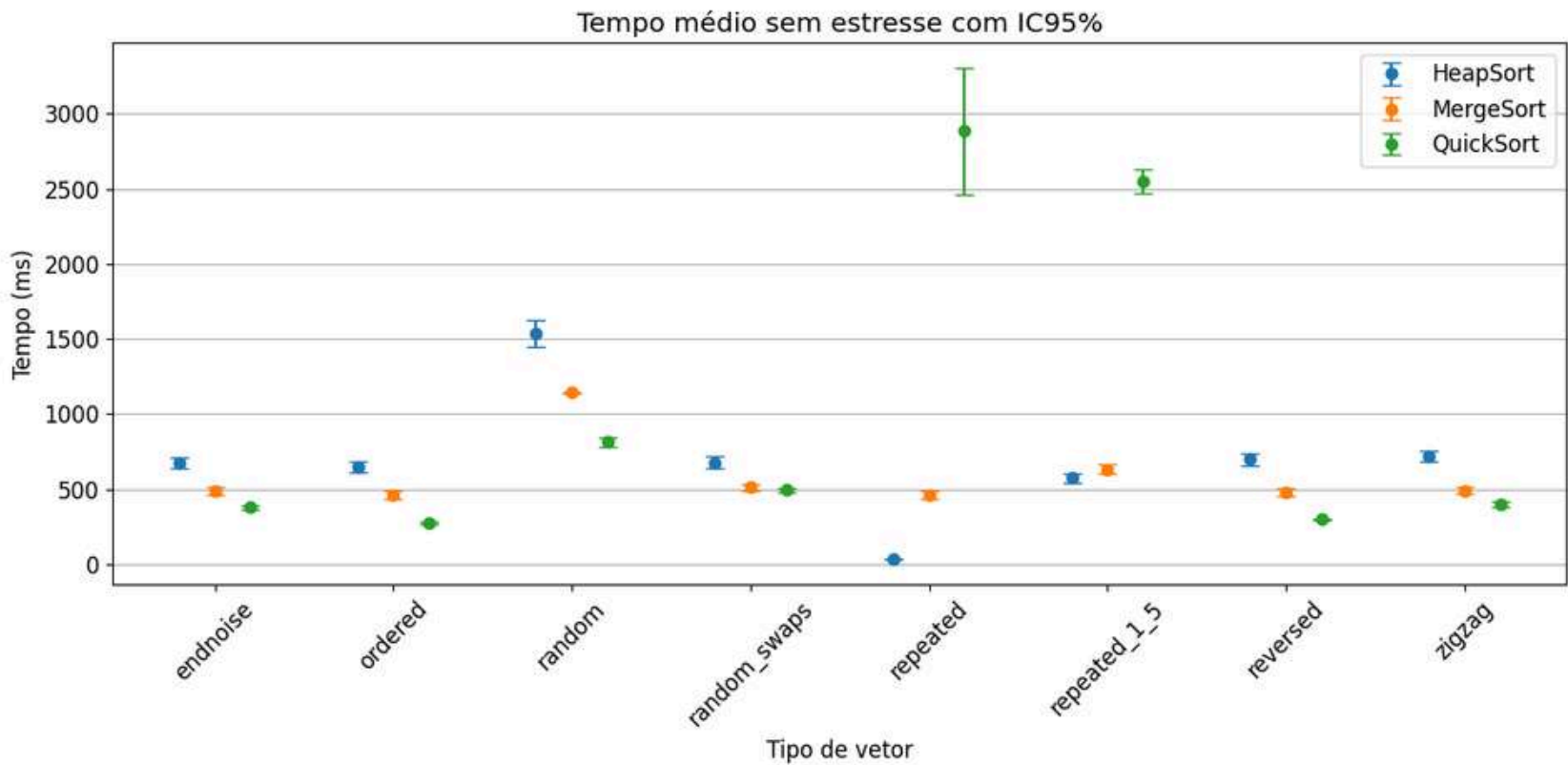


# ENERGIA CONSUMIDA

## ENERGIA



## TEMPO



## DEFINIÇÃO DO STRESS

### CPU

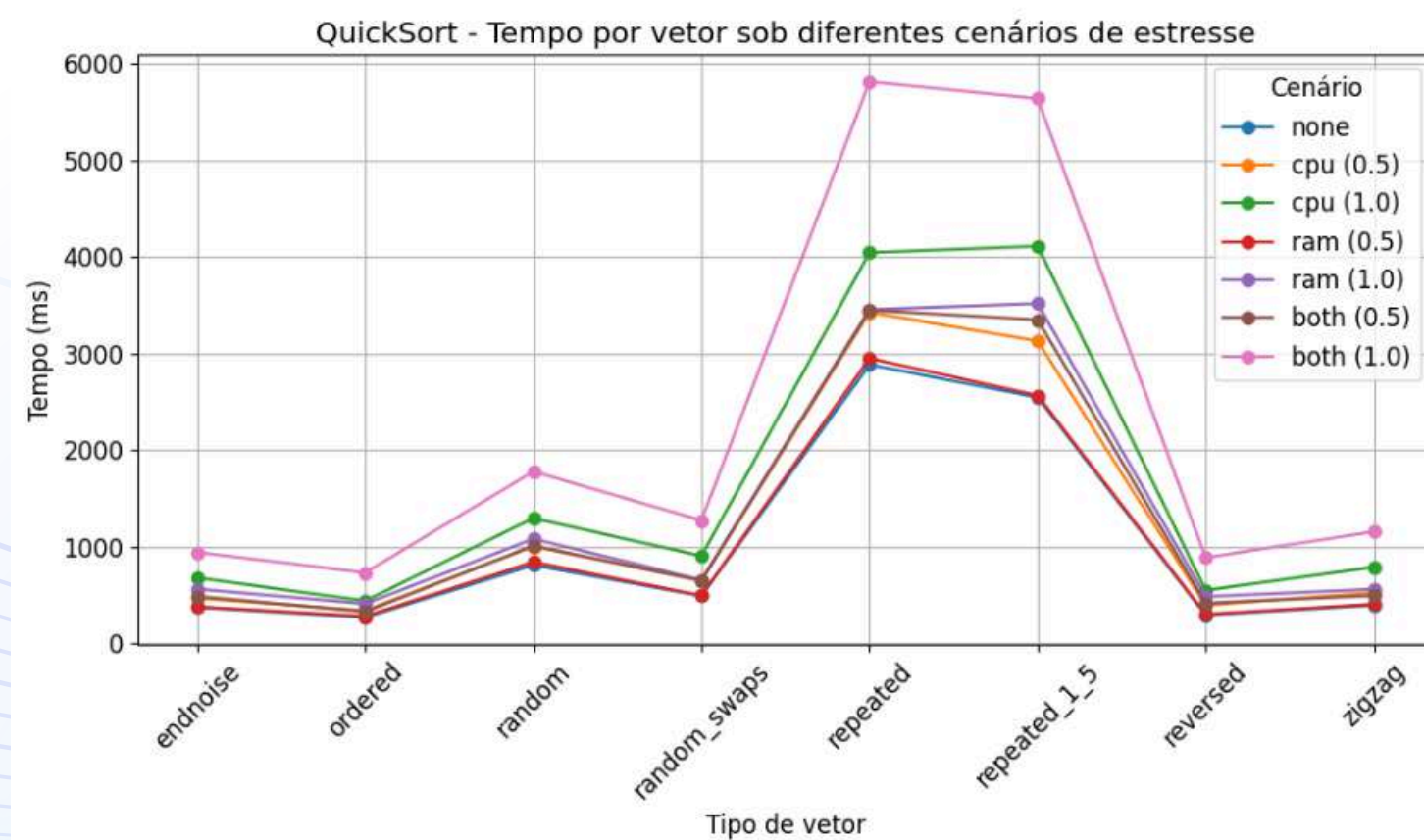
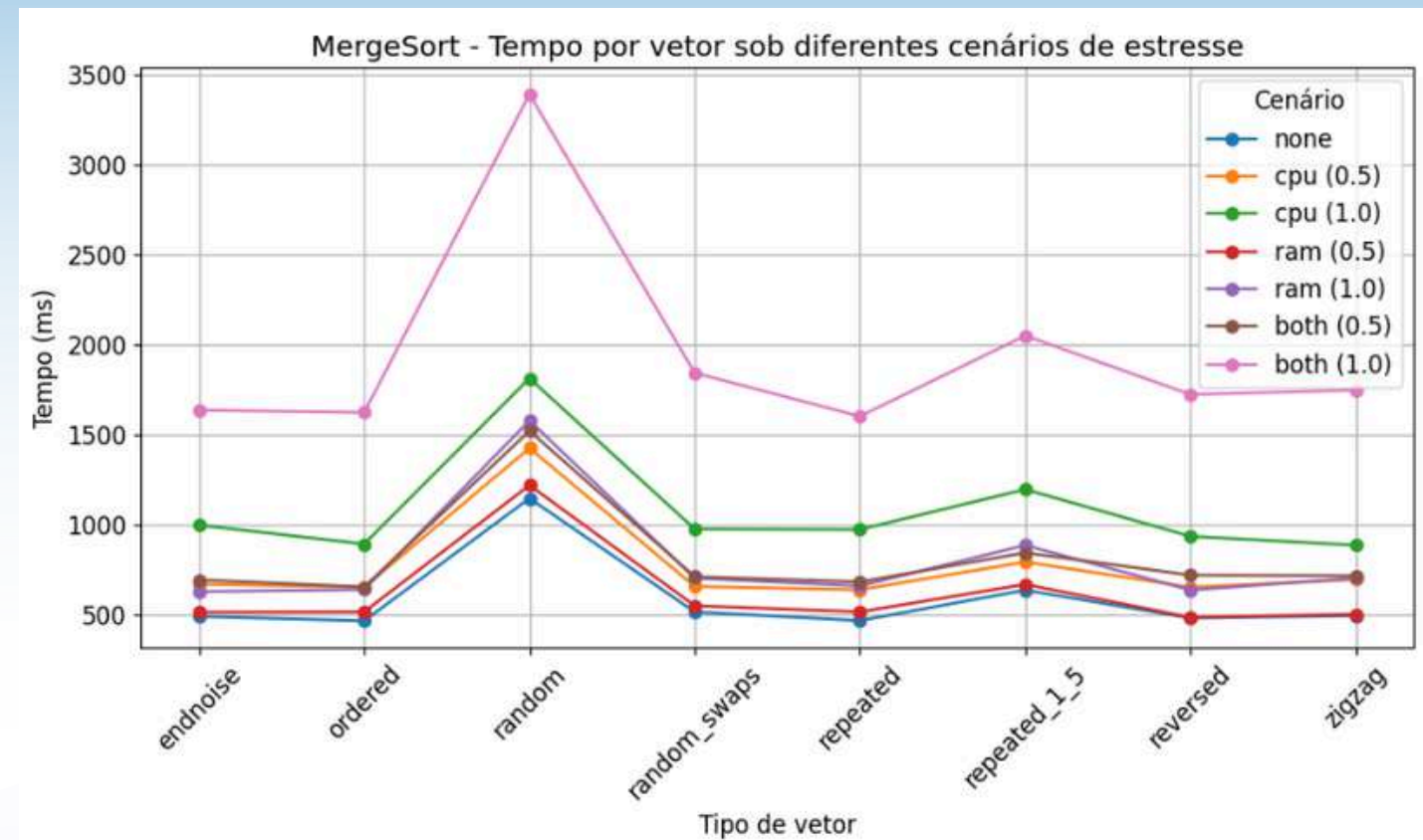
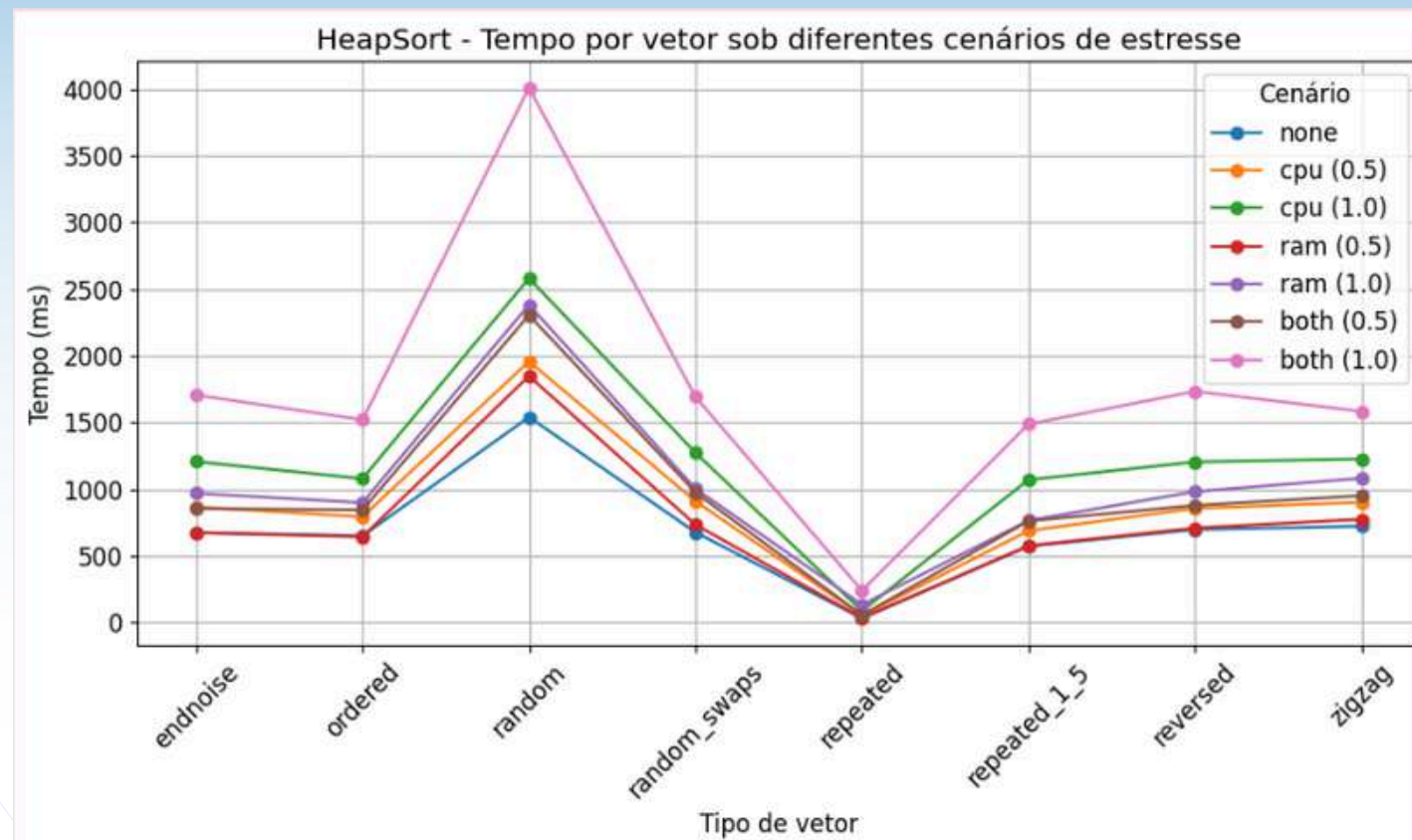
- 50%: 16 Threads com operações matemáticas em loop;
- 100%: 32 Threads com operações matemáticas em loop;

### RAM

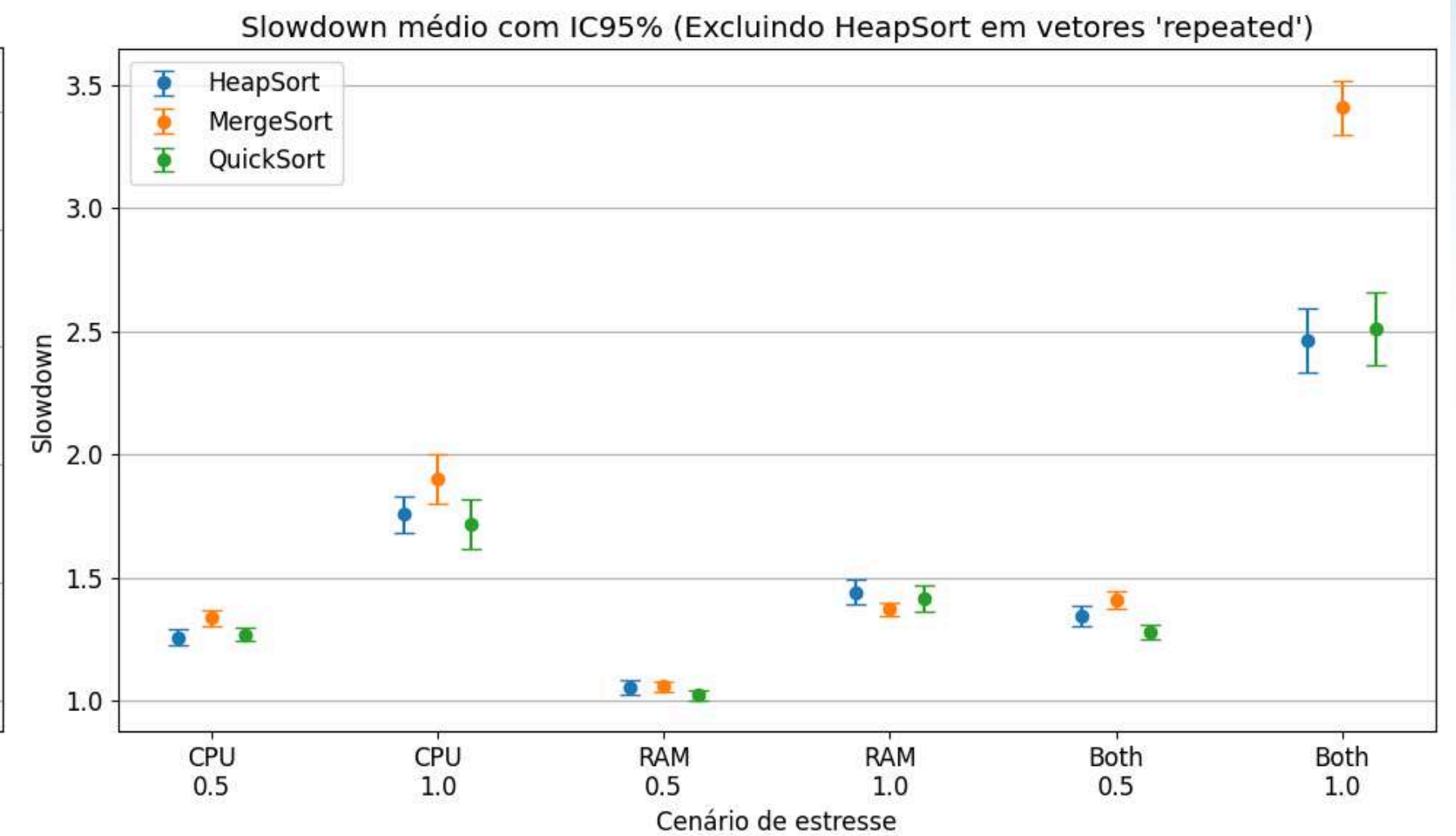
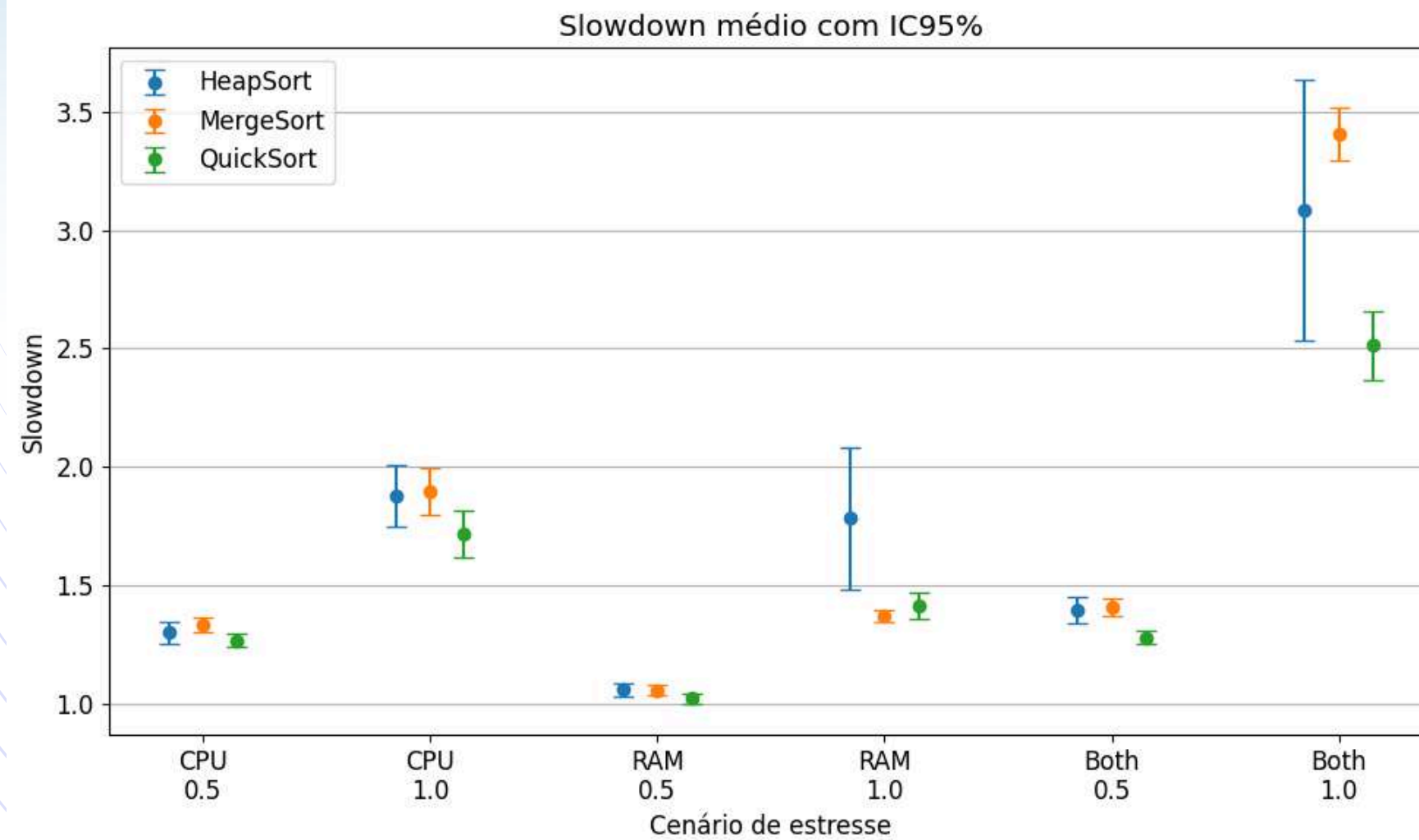
- 50%: 16GB alocados;
- 100% 32GB alocados;

PS: Custo Sistema Operacional: aprox. 5,4GB

# IMPACTO DO ESTRESSE



# IMPACTO DO ESTRESSE



## AMEAÇAS À VALIDADE

- Foi usado apenas um ambiente de simulação (Hardware e SO);
- Estresse não necessariamente reproduz casos reais;
- Sistema operacional pode influenciar medições;
- Conjuntos de dados limitados;
- Apenas 3 implementações específicas em C++;
- Algoritmos podem carecer de otimizações;
- Apenas 2 níveis de estresse;
- Limite de memória RAM sob estresse gera comportamento imprevisível;

## CONCLUSÃO

- Quicksort apresentou melhores resultados na maioria dos casos e a menor queda de desempenho no vetor aleatório, porém pior performance em valores repetidos;
- Heapsort destacou-se em casos com vetores de valores repetidos exaustivamente e queda de performance sob estresse muito próxima do Quicksort;
- Mergesort apresentou a maior consistência sem estresse porém pior queda de eficiência sob estresse;
- Otimização de código: Quicksort apresentou melhores resultados exceto quando há muitos dados repetidos, onde Heapsort se destaca. Diferentes níveis de estresse não modifica essa escolha como otimizada.

## USO DE IA

### ChatGPT

- Passo-a-Passo;
- Descrição do código desejado, trechos de códigos, dúvidas e comportamento durante testes;
- Definir problema → consulta à IA → Análise → Testes → Mudanças manuais → Integração;
- Leitura, compilação local e testes experimentais;
- Códigos .cpp, CMAKE, scripts, PANDAS
- e correção dos notebooks;

### Claude

- Leitura de medições do sistema;
- Correções de bugs e interpretação de erros de compilação;